



Chapter 1: Getting Started with the Windows App SDK and WinUI

Windows desktop applications have always played a critical role in the enterprise and productivity space. No matter what you do in your day-to-day job, if you are using a computer in a professional capacity, you are using one or more desktop applications to get your job done – Visual Studio, Office, and Photoshop are just a few of the most famous examples. And let's not forget many of the line-of-business applications that we might use in our everyday jobs to perform tasks such as submitting expense reports or creating financial reports.

Web and mobile devices have certainly changed the ecosystem, and many of these tasks can be performed everywhere nowadays. But this doesn't mean that desktop applications aren't relevant anymore. They still play a critical role in our productivity, thanks to better performance, which makes them the best choice for heavy tasks such as video rendering or graphic design. They are optimized for mouse and keyboard, which is still the primary input method for scenarios such as coding, data entry, and data analysis. They can be deeply integrated with every type of external hardware, such as barcode readers, scanners, and blood sample testers.

For all these reasons, Microsoft continues to heavily invest in the Windows desktop space to provide developers the best platform and tools to create powerful experiences for their customers. And with the release of Windows 11, there's a renewed interest among developers to delight their users with applications that take advantage of the latest innovations in the platform.

In this chapter, we're going to explore the following topics:

- What the Windows App SDK is and how it compares to the other existing development platforms for Windows
- The role of the new .NET runtime
- Choosing the right deployment model for your application
- Creating your first Windows App SDK project
- Managing the relationship between the Windows App SDK and Windows
- Building libraries and components

These topics will set the stage for you to get started with the Windows App SDK and **WinUI** (the short name for **Windows UI Library**), which will be useful for the next chapters.

Technical requirements

To build applications with the Windows App SDK, you will need the following:

- A computer with the latest version of Windows 10 or Windows 11.
- Visual Studio 2022 with the following workloads:
- **Universal Windows Platform (UWP)** development
 - .NET desktop development
 - Desktop development with C++
- The Windows SDK version 2004 (build 19041) or later. This SDK will be installed with Visual Studio when you enable the UWP development workload.
- The .NET 6 SDK. This SDK will be installed together with Visual Studio when you enable the .NET Framework desktop development workload.
- The Visual Studio extension for the Windows App SDK (if you are using Visual Studio 2022 Update 1 or later, it will already be included).

The code for the chapter can be found at the following URL:

<https://github.com/PacktPublishing/Modernizing-Your-Windows-Applications-with-the-Windows-Apps-SDK-and-WinUI/tree/main/Chapter01>

A brief history of Windows UI platforms

Over the years, UI guidelines and paradigms have constantly shifted as hardware and platforms evolved. We moved from screens with 640 x 480 resolution to 4K or even 8K screens, from mouse and keyboard only to touch and digital pens. Consequently, Microsoft has created multiple UI platforms over time, with the goal of offering developers the opportunity to build modern applications; each of them represented the state of the art for the time when they were released.

The first platform was called **Microsoft Foundation Class Library (MFC)**, which was a C++ object-oriented UI library released by Microsoft in 1992. It was a wrapper around most of the Win32 and **Component Object Model (COM)** APIs. Thanks to MFC, developers were able to build UIs with the most common Windows controls and build complex interfaces made up of multiple windows, panels, and so on. MFC was a considerable success, and it's still heavily used today by many developers. The following screenshot shows the look and feel of a typical MFC application:

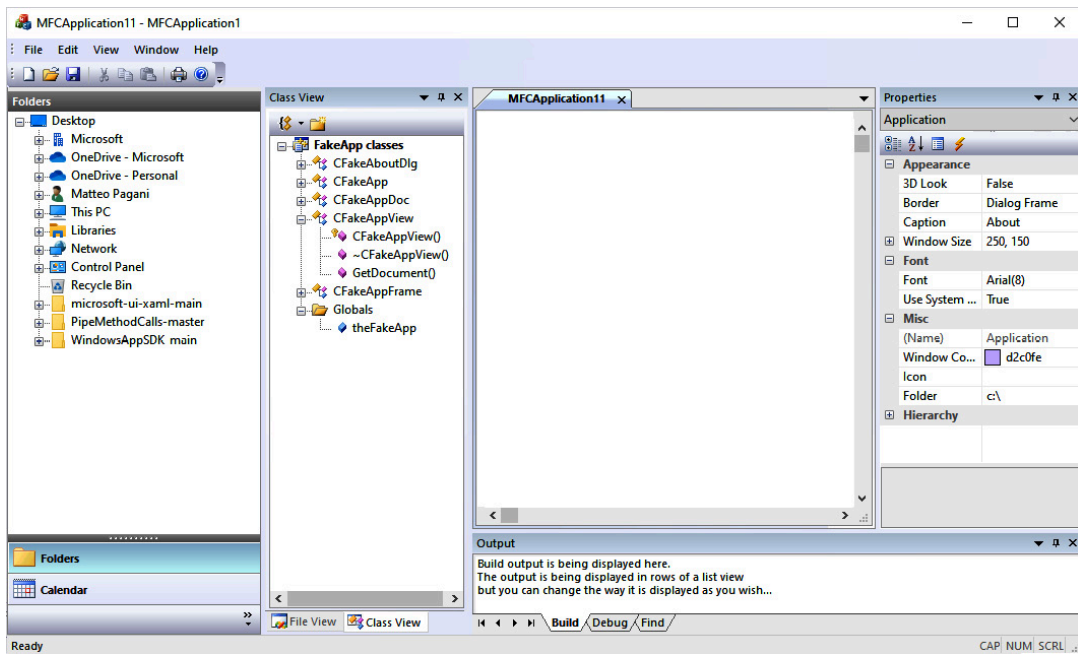


Figure 1.1 – A Windows application that uses MFC as a UI framework

However, as years passed by, it started to show limitations in supporting modern devices and features such as high-resolution screens and touch inputs. Additionally, it can be used only by C++ developers, while many developers over time have migrated to managed languages such as C#, which are easier to learn and support.

In 2002, Microsoft released the first version of .NET Framework with the goal of improving developer productivity. By running applications inside a virtual environment called **Common Language Runtime (CLR)**, developers could get out-of-the-box features such as security, memory, and exception handling that, in the past, needed to be manually managed. Additionally, by introducing languages such as C# and VB.NET, Microsoft reduced the learning curve required to master a programming language and start building software. As part of .NET Framework, Microsoft included a platform to build Windows desktop applications called **Windows Forms**. It's an event-driven platform, which makes it easier to build complex applications by wrapping the existing Windows UI common controls and Windows APIs in managed code. The development experience is mostly UI-based – developers create UIs with a visual designer by dragging and dropping the available controls inside a window. Then, they can write code that reacts to

the events exposed by the various controls, such as the click of a button or the selection of an item from a list. The following screenshot shows the development experience provided by Visual Studio to build Windows Forms applications:

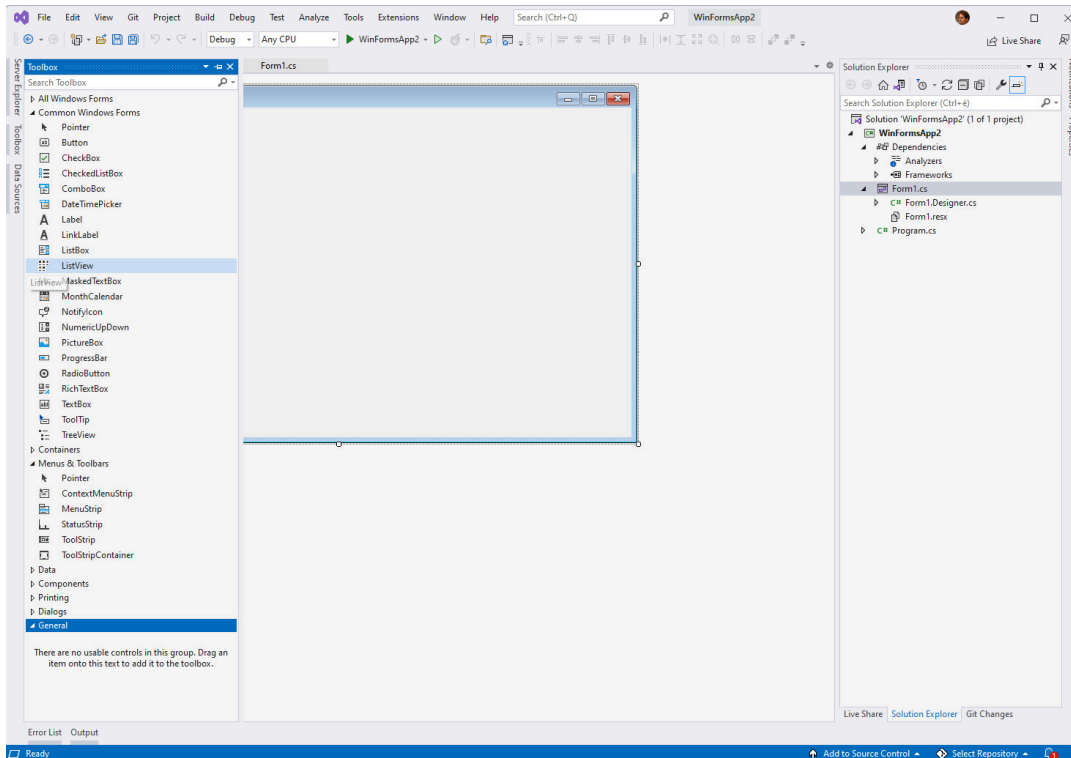


Figure 1.2 – The Windows Forms designer in Visual Studio

The platform kept evolving across the various releases of .NET Framework, until it reached full maturity with version 2.0.

With the release of .NET Core 3.0, Windows Forms has been integrated into the modern .NET development stack for the first time. This choice has enabled developers to access all the latest enhancements in the platform, such as newer versions of the C# language, performance improvements, or the latest Windows APIs. However, when it comes to building the UI, it still lacks many of the features you would expect from a modern platform, such as support for responsive layouts and new input experiences.

In 2006, as part of the release of .NET Framework 3.0, Microsoft unveiled **Windows Presentation Foundation (WPF)**, the next evolution

of the Microsoft UI platform. WPF introduced, for the first time, features that are still used today by modern UI platforms (including the Windows App SDK), such as **XAML** (which stands for **Extensible Application Markup Language**), binding, and dependency properties. WPF still supports building the UI with a designer, but it isn't as essential as it was for Windows Forms. WPF, in fact, decouples the UI from the business logic by describing the UI with XAML, an XML-based language. Additionally, WPF added support for features such as 2D/3D rendering, hardware acceleration, animations, and vector graphics. As with Windows Forms, .NET Core 3.0 welcomed WPF as a first-class citizen in the new development platform, enabling WPF developers to get access to the latest versions of runtimes, languages, and developer tools. Compared to Windows Forms, WPF is a more robust UI platform, capable of delivering more modern experiences, as you can see in the following screenshot:

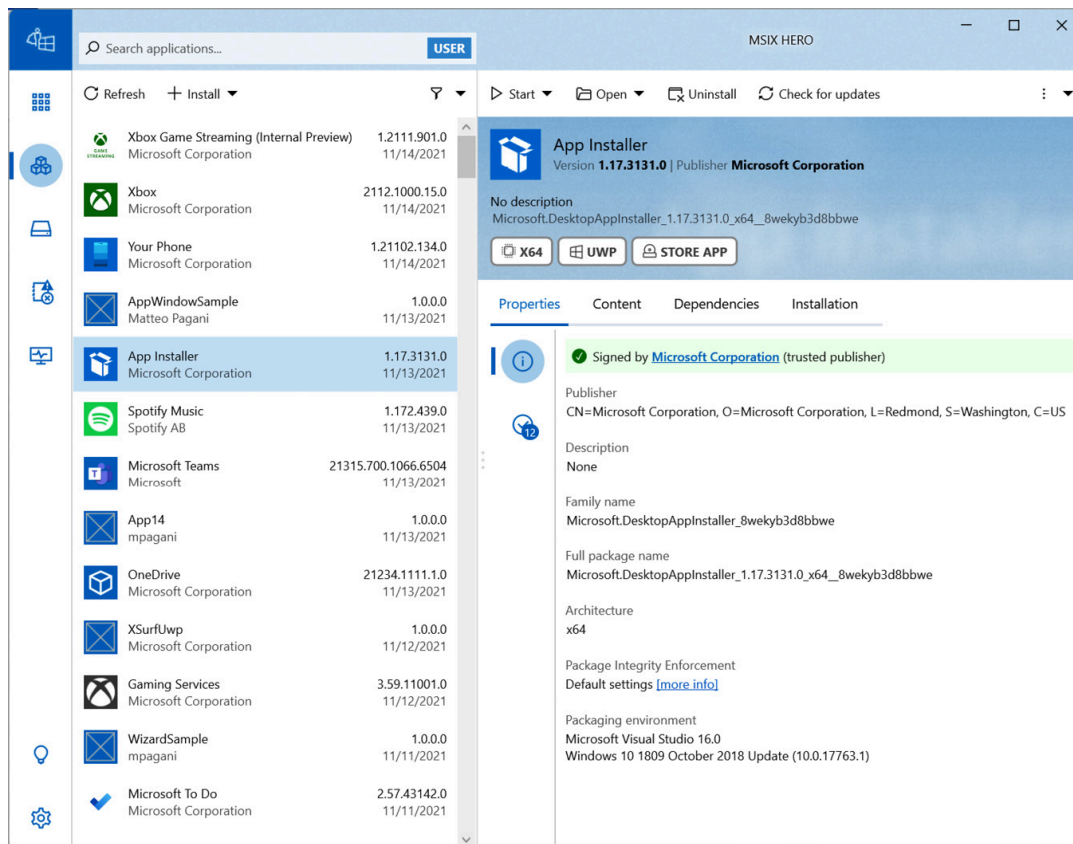


Figure 1.3 – MSIX Hero is a good example of an application that delivers a great user experience by using the WPF capabilities

However, it still has limitations, such as poor support to high **Dots-Per-Inch (DPI)** devices, touchscreens, digital inking experiences, and accessibility.

In 2015, with the release of Windows 10, Microsoft released UWP, which is an extension of Windows Runtime that was introduced in Windows 8. UWP is a modern development platform that enables developers to build secure and robust applications that run inside a sandbox; it gives access to all the new features added in Windows 10, such as tiles, notifications, and Windows Hello; it's based on a new UI platform called Fluent Design, which offers built-in support to responsive layout, touch and digital pen, accessibility, and so on. Many built-in Windows applications, such as **Microsoft Store**, are built with UWP and WinUI:

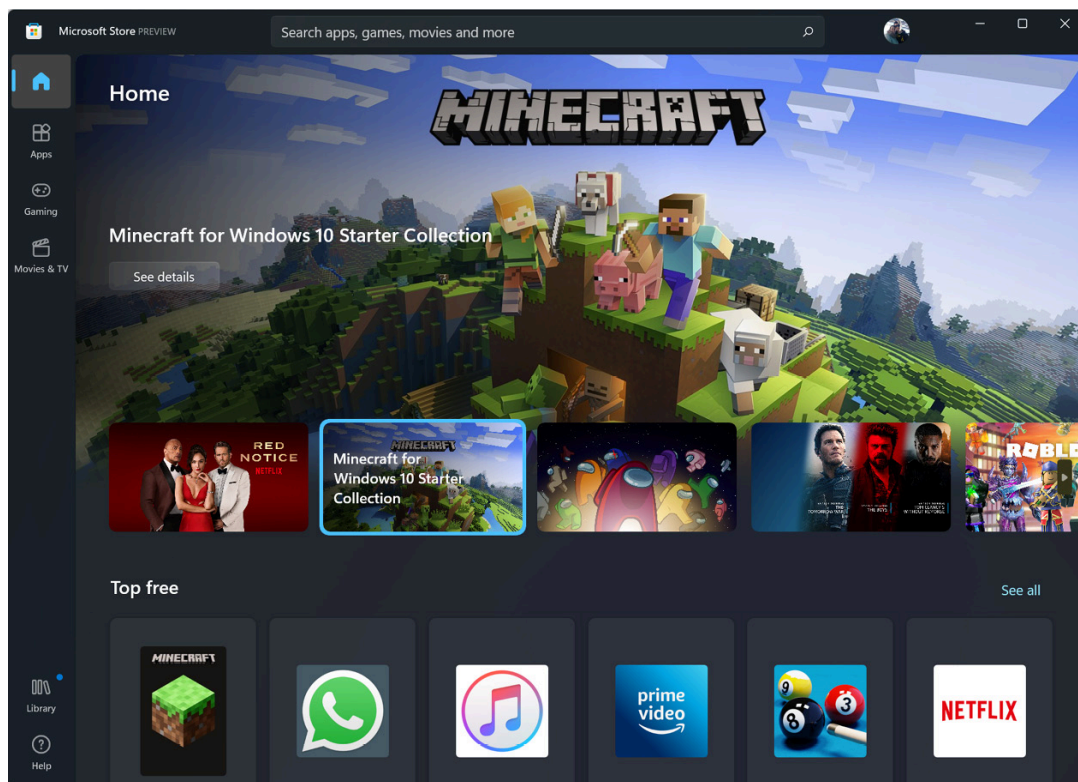


Figure 1.4 – Microsoft Store in Windows is a UWP application

In the first releases of UWP, the UI framework (like all the other development APIs) was built in the operating system. Over time, this approach created multiple challenges, both to Microsoft and developers:

- Despite Windows 10 adopting a much more aggressive update strategy compared to prior versions, by releasing two upgrades per year, it still forced the development team to address issues and add new UI controls and features only twice per year.
- If a developer wanted to use the new UI controls or features added to the latest version of Windows 10, all their users had to upgrade their machines to that version as well. This was a challenge, especially in enterprise environments, where the upgrade pace is slower than in the consumer world.

To overcome these challenges, in October 2018, Microsoft released the first public release of the Windows UI library, called **WinUI 2.0**. With this release, Microsoft detached most of the UI controls and features from the operating system and moved them inside a library, which is distributed as a NuGet package. The library enabled Microsoft to release more frequent updates (the current life cycle is four releases per year) and developers to get access to the latest UI enhancements without forcing their user base to upgrade to the latest Windows 10 version.

Introducing the Windows App SDK and WinUI 3.0

Now that we have learned a bit of the history of the Microsoft development platform for Windows, we can better understand how the Windows App SDK fits into the story. We've learned that, over time, Microsoft has released new UI platforms with the goal of being state of the art at the time of release. However, UWP introduced a few challenges, especially for developers building line-of-business applications. UWP is a great fit for many consumer scenarios – the sandbox enables an application to run more safely, since it has limited access to critical Windows features, such as a registry or a full filesystem; thanks to a rich capabilities system, the user is always in control of which features

of the device (a webcam, a microphone, location, and so on) an application can access; and thanks to a modern life cycle, applications are more respectful of the CPU, memory, and battery life of a device. In many cases, however, enterprises require deeper control and flexibility – applications need to always run, even when they are minimized in the taskbar; they need to interact with custom hardware devices and retrieve information from the Windows Registry.

Also, Windows Runtime, the application architecture behind UWP, introduced a few challenges. This modern runtime offers a lot of benefits: it's built in C++, which means it offers great performance; it's based on asynchronous patterns, which help developers to build applications that are fast and responsive; and through a feature called projections, it enables developers to consume Windows Runtime APIs from multiple languages, such as C++, C#, and Rust. However, since it's a significantly different platform compared to the ones already on the market (such as .NET), developers who have heavily invested in C++, Windows Forms, and WPF are able to reuse little or no code when porting their applications to UWP. This requirement created a lot of friction, since in the enterprise ecosystem, it's easy to find desktop applications with decades of development that are very hard, if not impossible, to port to UWP without rewriting them from scratch. The outcome is that even if developers loved Fluent Design and the new features introduced in Windows 10, most of them weren't able to take advantage of them.

In late 2019, Microsoft announced the first milestone to overcome these challenges – WinUI 3.0. Earlier versions of WinUI already started to lift most of the UI controls and features from the operating system, but the library was still targeting only UWP. With the 3.0 release, instead, Microsoft lifted the whole UI framework from the operating system, enabling other development platforms such as .NET to start taking advantage of the new UI library.

Thanks to WinUI 3.0, developers can now build modern applications and experiences but, at the same time, leverage familiar development platforms such as .NET or C++; reuse most of the investments they did building Windows Forms, WPF, or C++ applications; and use popular NuGet packages that they had already adopted in Windows Forms and WPF applications. Additionally, since applications using WinUI 3.0 are no longer dependent on UWP, but instead run as classic desktop applications, many of the features that enterprise developers saw as a constraint (the sandbox, the life cycle optimized to reduce CPU, and memory usage) don't apply anymore.

Microsoft, during the 2020 edition of the Build conference, officially shared the next step of the journey by announcing **Project Reunion**, a new development platform with the goal to provide developers with the best of both worlds – familiar developer frameworks and languages (such as .NET, C#, and C++) and the ability to use all the Windows features. WinUI 3.0 was incorporated into this new platform, and it became its first and most important building block.

Project Reunion continued to evolve over time by gradually bringing new features that were exclusively a part of UWP, such as push notifications and activation contracts. On June 24, 2021, when Microsoft revealed to the world Windows 11, the Windows development team announced the official name of Project Reunion – **the Windows App SDK**.

The Windows App SDK and WinUI 3.0 have the ambitious goal of becoming the new reference UI platform for all Windows developers, regardless of their background. In fact, the Windows App SDK targets both C++ developers (who, before the Windows App SDK, were stuck on MFC as the UI framework offered by Microsoft) and .NET developers (who can reuse their existing skills but target a more modern UI framework than Windows Forms and WPF).

The Windows App SDK introduced the following advantages for developers:

- **Consistent support:** Windows and its development platform continue to evolve over time, and developers must rely on techniques such as adaptive coding to keep a single code base and use a specific feature only if the application is running on a version of the operating system that supports it. Over time, this approach can add complexity, so Microsoft decided to create the Windows App SDK with built-in down-level support. An application that uses the Windows App SDK can run from Windows 11 down to Windows 10, version 1809. Newer features will be automatically enabled if the application is running on the proper version of the operating system. An example of this feature is Mica, a new material introduced in Windows 11 to enrich the background of an application. If you configure your application to use it, Mica will automatically light up if the application is running on Windows 11, while it will automatically fall back to a solid color on Windows 10.
- **A faster release cadence:** Since the Windows App SDK is a library, Microsoft can ship updates faster, without being aligned to the release cycle of Windows.
- **A unified API surface:** Historically, the number of Windows features you could use in your application depended on your framework of choice. For example, many Windows 10 features were supported only by UWP apps. The Windows App SDK unifies access to Windows APIs so that, regardless of the app model you choose, you will be able to use the same set of Windows features.

The Windows App SDK and WinUI are considered the future of the Windows developer platform. UWP is now considered a stable and mature platform that will continue to receive security updates and be supported. However, all the investments for new features and scenarios will be focused on the Windows App SDK, making it the best choice to build future-proof Windows applications.

This book is dedicated to C# developers, and it will cover the usage of the Windows App SDK with classic desktop applications based on the .NET runtime. However, every concept you are going to learn can also be applied to C++ applications.

The role of the new .NET runtime

.NET Framework was introduced by Microsoft in 2002 with the goal to provide a better development experience and an easier learning curve for developers to build applications for Windows. These principles still hold true today, but the development landscape has changed significantly since 2002. New platforms have appeared, and Windows, in the server and cloud space, isn't the leading platform anymore; scenarios such as containers and microservices require a new level of optimization and performance, especially in key areas such as networking or filesystem access. Open source is now the new standard to release and evolve development platforms in collaboration with the community.

In 2014, Microsoft announced the first version of .NET Core, a new development platform based on the same principles of .NET Framework but open source, cross-platform, and lighter. The first versions of .NET Core were focused on key scenarios such as cloud services and web applications. With the release of .NET Core 3.0, Microsoft has started to invest also in the desktop ecosystem, by making Windows Forms and WPF open source and welcoming them into the new runtime.

However, the .NET ecosystem was still fragmented – if web and desktop applications could run on .NET Core, many developers were still using the full .NET Framework to build their solutions; Xamarin (the cross-platform framework to build Android and iOS apps with C#) and Blazor (the new platform to build client-side web applications using C# instead of JavaScript) were still based on Mono, the original open source implementation of .NET Framework. This fragmentation created multiple challenges over time and required the team to create so-

solutions such as Portable Class Libraries and .NET Standard to enable developers to share code and libraries across different projects.

With .NET 5 (released in November 2020), Microsoft started an ambitious project to unify the entire .NET ecosystem. Instead of having multiple implementations of the framework for different workloads, .NET 5 started a unification journey, with the goal of supporting all the platforms and devices on the market with a single runtime and base class library. .NET 6 is going to complete this journey with a future update by abandoning Mono and bringing Xamarin (now known as .NET for Android and .NET for iOS) inside the family.

The new .NET runtime has also introduced a more predictable release plan, which makes it easier for developers to plan the adoption of one version or another:

1. .NET 5 was released in November 2020.
2. A new version will be released around the same time frame each year – for example, NET 6 was released in November 2021, and .NET 7 will be released in November 2022.
3. The releases with an even version number (.NET 6, .NET 8, and so on) are marked **Long-Term Support (LTS)**. These releases will be supported for 3 years, so they're a better fit for projects that need stability.
4. The releases with an uneven version number (.NET 5, .NET 7, and so on) are supported for 6 months until the release of the next version – for example, .NET 5 will go out of support in May 2022, 6 months after the release of .NET 6. You can check the most up-to-date support policy at <https://dotnet.microsoft.com/en-us/platform/support/policy/dotnet-core>.

By adopting a more frequent release cycle, the .NET team can be more agile and reduce the chances of introducing breaking changes between one version and the other. Even if you decide to adopt a .NET

runtime that is not marked as LTS, in most cases, the transition to the next release will be smooth, with no or very few code changes needed.

Consequently, .NET Framework reached the end of the journey with the release of .NET Framework 4.8. This framework is still popular today, and as such, Microsoft doesn't have any plan of deprecating it. The current .NET Framework support policy is tied to the Windows one, so .NET Framework will continue to be supported for a long time. However, it won't receive any new features or major updates. If you're a developer building Windows apps that are continuously upgraded and evolved, the suggested path forward is to migrate your .NET Framework applications to the new .NET runtime so that you can take advantage of the constant evolution of the platform.

For all these reasons, the new .NET runtime plays a critical role in our journey. Since the Windows App SDK is the future of Windows development, its evolution is deeply connected with the new .NET runtime. For example, the first preview releases of the Windows App SDK were targeting .NET 5, but the platform is now aligned with the most recent .NET 6 release.

Additionally, all the features exposed by the Windows App SDK don't support the old .NET Framework, but they require the new .NET runtime.

Exploring the Windows App SDK

The Windows App SDK has currently reached version 1.0, and it includes the following features:

- WinUI 3
- Text rendering
- Resource management
- App life cycle
- Power state notifications

- Windowing
- Push notifications (preview)

The second section of this book will be focused on WinUI 3 and will guide you toward a modernization journey, with the goal of evolving your application from your existing UI framework (Windows Forms, WPF, or UWP) to embrace the latest innovation in Fluent Design.

This feature of the Windows App SDK is currently supported only by new apps that target WinUI 3. This means that this modernization journey will help you to reuse most of your existing code and libraries (thanks to the .NET runtime), but you won't be able to gradually move the UI of your Windows Forms or WPF application. You will have to use one of the new Visual Studio templates to create a new .NET application based on WinUI. Microsoft is working to bring a technology called XAML Islands to WinUI 3, which will instead enable developers to gradually migrate the UI of their applications by mixing controls from WinUI with controls from the existing UI frameworks (Windows Forms and WPF).

The third section, instead, will be focused on the developer platform – you will learn how to integrate the other features offered by the Windows App SDK, plus many other new APIs (such as geolocation and machine learning), which, previously, were available only to UWP applications. These features, unlike WinUI 3.0, can be easily integrated into existing desktop applications built with WPF and Windows Forms, as long as they have been migrated to at least .NET 5.

The Windows App SDK is distributed in three different channels:

- **Stable:** The releases distributed through this channel are supported in production environments. They include the latest stable and tested bits.
- **Preview:** The releases distributed through this channel include preview features that will be added in the next stable release.

Being a preview, they aren't supported in production scenarios, since there might be breaking changes when the stable version gets released.

- **Experimental:** The releases distributed through this channel include experimental features, which might be discarded or completely changed in the next stable release. They aren't supported in production environments, since the included features might not even see the light.

All the features described in this book are included in the stable channel, except for push notifications, which are still in preview.

With an understanding of the Windows App SDK, our next step is to learn how to choose between an unpackaged and packaged deployment model. We will do that in the next section.

Choosing the right deployment model

We'll talk in more detail about how to deploy distributed applications that are using the Windows App SDK in [***Chapter 11***](#), *Publishing Your Application*, but it's critical to introduce at the beginning two important concepts, since they will influence the way you create an application – packaged and unpackaged.

Packaged applications adopt MSIX as a deployment technology. MSIX is the most recent packaging format introduced in Windows, which, compared to other deployment technologies such as MSI or ClickOnce, brings many benefits:

- A cleaner install and uninstall process, thanks to the usage of a lightweight container that virtualizes many of the critical aspects of Windows, such as the filesystem and the registry

- Built-in features such as automatic updates even in unmanaged scenarios (such as a website), bandwidth optimization, and disk space optimization
- Tampering protection, which prevents an app that has been improperly changed from running and damaging an operating system

Windows applications packaged with MSIX are described by a manifest, which is an XML file that holds information such as the name, the publisher, the version number, and the dependencies. The packaged approach is the best one for applications that use the Windows App SDK, since it simplifies many of the scenarios that we're going to see in this book, such as managing the framework dependency and using Windows APIs that require an identity.

However, as a developer, you might face scenarios where a packaged app doesn't fit your requirements:

- The container provided by MSIX is very thin, but there are still situations when it might interfere with the regular execution of your application due to the isolation of the registry and the filesystem.
- To work properly, the application must deploy, during the installation, a kernel driver, or it must apply some global settings to the computer, such as creating environment variables and installing a Windows feature. The isolated nature of MSIX doesn't make these kinds of scenarios a good fit.

Because of these cases, the Windows App SDK also supports unpackaged apps, which are applications that you can deploy the way you prefer by using manual copy deployment, adopting a traditional MSI installer, building a custom setup, or using a script. The way you deploy your application is deeply connected to the way you manage the dependency that your application has with the Windows App SDK.

Let's learn more in the next section.

Managing the dependency with the Windows App SDK

By detaching features and APIs from the operating system, the Windows team has gained a lot of benefits, such as the ability to add new features and fix issues outside the regular Windows update cycle. However, as developers, we cannot take for granted that the user who installs our application will have the Windows App SDK runtime installed on their machine. As such, it's our responsibility as developers to manage this dependency in the right way. This is one of the scenarios where the difference between adopting the packaged or unpackaged model is critical, since they adopt two different ways to deploy the dependency.

The Windows App SDK runtime is made up of the following components:

- **The framework package:** This package contains the binaries that are used by the applications at runtime.
- **The main package:** This package contains multiple out-of-process components that aren't included in the framework, such as background tasks, app extensions, and app services.
- **The singleton package:** This package includes a long-running process that provides access to features that are shared across all applications, such as push notifications.
- **Dynamic Dependency Lifetime Manager (DDLTM):** This package is used to control the life cycle of the Windows App SDK runtime. Thanks to this package, Windows will refrain from automatically updating the runtime if one or more unpackaged apps that use the Windows App SDK are running.

Each of these components is stored in its own MSIX package. Let's see now how to manage the deployment of the runtime based on the way you distribute your application.

Packaged apps

As previously mentioned, packaged apps have a few advantages compared to unpackaged apps. Packaged applications are described by a manifest, which includes a section called **Dependencies**. This section is already used today to manage the dependency on a specific version of Windows or the Visual C++ runtime.

The dependency with the Windows App SDK runtime can be easily declared in the same section by adding the following entry:

```
<PackageDependency
  Name="Microsoft.WindowsAppRuntime.1.0"
  MinVersion="0.319.455.0" Publisher="CN=Microsoft
    Corporation, O=Microsoft Corporation, L=Redmond,
    S=Washington, C=US" />
```

Thanks to this configuration, if the application is deployed to a machine that doesn't have the runtime installed, Windows will automatically download it from the Microsoft Store and install it system-wide so that future applications that you might install won't need to download it again.

In most cases, you won't have to manually include this entry. In fact, when you install the Windows App SDK NuGet package in your application, Visual Studio will add a special build task that will add this entry in the manifest for you every time you generate a new MSIX package.

However, there's still a manual requirement you have to take care of. As mentioned in the previous section, the Windows App SDK applications are dependent not just on the framework itself but, based on the features they use (for example, background tasks or push notifications), they might also need the main package and the singleton package. MSIX doesn't support having regular packages as dependencies, which means that when you deploy a packaged Windows App SDK ap-

plication, the framework is automatically deployed if missing, but the other components aren't. However, these packages are included inside the framework and, as such, can be automatically deployed by using the **DeploymentManager** API, which belongs to the **Microsoft.Windows.ApplicationModel.WindowsAppRuntime** namespace. Thanks to this API, you can check whether one or more of the components are missing on the system and install them if necessary. This is an example implementation of the **OnLaunched()** method of a Windows App SDK application, which is executed when the application starts:

```
protected override void OnLaunched
    (Microsoft.UI.Xaml.LaunchActivatedEventArgs args)
{
    var status = DeploymentManager.GetStatus();
    if (status.Status ==
        DeploymentStatus.PackageInstallRequired)
    {
        DeploymentManager.Initialize();
    }
    m_window = new MainWindow();
    m_window.Activate();
}
```

Thanks to the **GetStatus()** method, we can detect the status of the various components on the system. If one or more of them is missing (which is represented by the **PackageInstallRequired** value of the **DeploymentStatus** enumerator), we can call the **Initialize()** method to perform the deployment.

Unpackaged apps

For unpackaged apps, Microsoft provides an installer called **WindowsAppRuntimeInstall.exe**, which automatically detects the CPU architecture of a system (x86, x64, or ARM64) and installs system-wide

the MSIX packages that compose the runtime – framework, main, and DDLM.

The installer also supports a **--quiet** parameter, which enables a silent installation with no user interaction and output messages. All the technologies and tools to create setup programs (or even just a PowerShell script) support the possibility of launching an executable as part of the installation, so it's easy to configure your installer to silently launch the **WindowsAppRuntimeInstall.exe** executable before or after the deployment of the main application, and before the whole installation process is completed.

Compared to a packaged application, you won't need to use the deployment APIs here, since the **WindowsAppSDKInstall** tool will take care of installing all the required packages on the system.

Upgrading the Windows App SDK runtime

The Windows App SDK runtime will continue to evolve over time, and as such, it's important for a developer to understand how future updates will be managed.

Updates can be delivered in two ways:

- If the machine has access to Microsoft Store, they will be automatically downloaded and installed.
- If the machine doesn't have access to Microsoft Store, it will be up to the application to include a newer version of the Windows App SDK runtime installer.

To control the way updates will be installed, the Windows App SDK has adopted the Semantic Versioning rules. By reading the official website (<https://semver.org/>), we learn that the version number is defined by three numbers, MAJOR.MINOR.PATCH, which stand for the following:

- MAJOR is incremented when you make incompatible API changes.
- MINOR is incremented when you add new functionality but with backward compatibility.
- PATCH is increased when you make bug fixes that are backward-compatible.

The runtime installed on a machine will be automatically updated only if the MINOR or the PATCH revision changes. If a new release increases the MAJOR number, instead, it will be installed side by side with the existing ones. This makes sure that newer versions of the runtime that might include breaking changes won't replace the existing ones, causing your applications to stop working properly.

Thanks to the DDLM component of the runtime, Windows will keep the old instance of the runtime installed as long as there's at least one running application that is using it. Once the application is closed, the previous version of the runtime will be uninstalled, and at the next re-launch, the app will start using the updated version.

Now that we understand how to manage the Windows App SDK dependency in our application, we're ready to start creating our first application.

Creating the first Windows App SDK project

When it comes to adopting the Windows App SDK, there are two options:

- Start with a new project using one of the available Visual Studio templates.
- Integrate the Windows App SDK into an existing application. Since this book is dedicated to C# developers, we'll focus on Windows Forms and WPF.

We will discuss each possibility in the next sections.

A new packaged WinUI C# application

Once you have installed the Windows App SDK extension for Visual Studio, you'll get a few templates to create new projects. The starting point for a Windows App SDK application is called **Blank App, Packaged (WinUI 3 in Desktop)**, as shown in the next screenshot. Why WinUI? Because it's the only feature of the Windows App SDK that can't be integrated into existing applications, so you will likely start with a new project if you want to create a Windows application that uses WinUI as the UI framework:

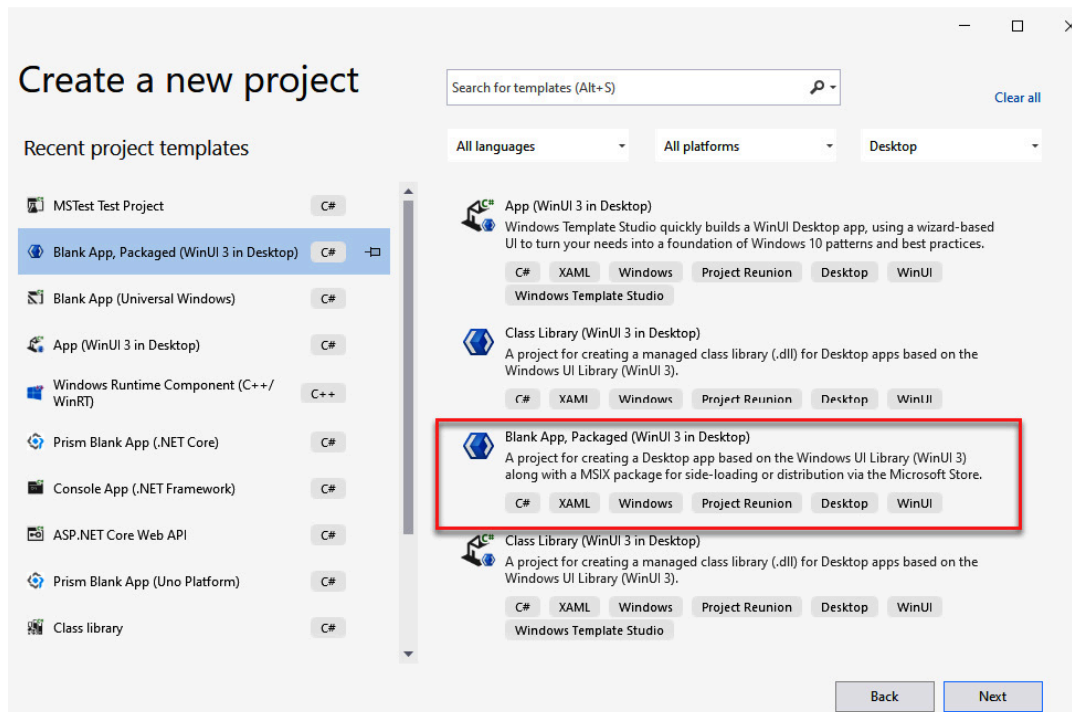


Figure 1.5 – The template to create a new packaged WinUI 3 application

The template will create a project that contains all the basic building blocks of a WinUI application. Let's take a look at the structure in more detail.

The template will create a project as shown in the following screenshot:

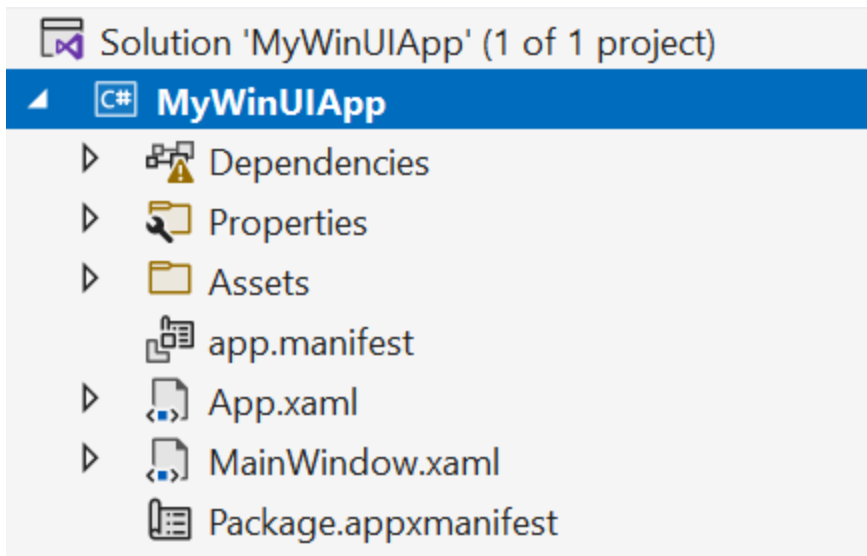


Figure 1.6 – A new solution for a packaged app that uses the Windows App SDK and WinUI 3

This is the project that contains the actual application, and that's where you're going to write most of your code.

One of the key differences between .NET Framework and the new .NET runtime is the adoption of a new project format, called SDK style. This format makes it quite easy to read and change the XML that describes the project. Visual Studio still supplies a more human-friendly UI (which you can see by right-clicking on the project and choosing **Properties**), but it isn't a must-have feature anymore.

This is what the project file of an application that uses the Windows App SDK looks like (you can see it by simply double-clicking on the project's name in Solution Explorer):

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <OutputType>WinExe</OutputType>
    <TargetFramework>net6.0-
      windows10.0.19041.0</TargetFramework>
    <TargetPlatformMinVersion>10.0.17763.0
    </TargetPlatformMinVersion>
```

```
<RootNamespace>MyApplication</RootNamespace>
<ApplicationManifest>app.manifest</ApplicationManifest>
<Platforms>x86;x64;arm64</Platforms>
<RuntimeIdentifiers>win10-x86;win10-x64;
  win10-arm64</RuntimeIdentifiers>
<PublishProfile>win10-$(Platform).pubxml
  </PublishProfile>
<UseWinUI>true</UseWinUI>
<EnablePreviewMsixTooling>true</EnablePreview
  MsixTooling>
</PropertyGroup>
<ItemGroup>
  <AppxManifest Include="Package.appxmanifest">
    <SubType>Designer</SubType>
  </AppxManifest>
</ItemGroup>
<ItemGroup>
  <Content Include="Assets\SplashScreen.scale-
200.png" />
  <Content Include="Assets\LockScreenLogo
    .scale-200.png" />
  <Content Include="Assets\Square150x150Logo
    .scale-200.png" />
  <Content Include="Assets\Square44x44Logo
    .scale-200.png" />
  <Content Include="Assets\Square44x44Logo
    .targetsized-24_altform-unplated.png" />
  <Content Include="Assets\StoreLogo.png" />
  <Content Include="Assets\Wide310x150Logo
    .scale-200.png" />
</ItemGroup>
<ItemGroup>
  <PackageReference Include="Microsoft.WindowsAppSDK"
    Version="1.0.0" />
```

```
<PackageReference
  Include="Microsoft.Windows.SDK.BuildTools"
  Version="10.0.22000.196" />
<Manifest Include="$(ApplicationManifest)" />
</ItemGroup>
</Project>
```

Let's take a look at the key properties:

- **TargetFramework:** This defines which flavor of .NET we want to use. In our scenario, we need to use the specific workload that gives us access to the Windows Runtime ecosystem, which is **net6.0-windows10.0.19041.0**.
- **Platforms:** Since the Windows App SDK is a native layer of APIs, it can't be used in applications that are compiled for the generic *Any CPU* architecture. As such, our project must explicitly declare the architectures we can support, which, in the case of the Windows App SDK, are x86, x64, and ARM64.
- **RuntimeIdentifiers:** The .NET runtime supports a deployment model called self-contained, in which the runtime itself is bundled together with the application. This helps to reduce the manual steps that a user must take to satisfy all the required dependencies. By default, a packaged Windows App SDK application uses this configuration so that you can generate a standalone package, which includes everything the user needs to run it.
- **UseWinUI:** Every .NET project for a Windows application specifies which is the used UI framework so that .NET can load the proper libraries. Since we're building a WinUI application, the property to use is **UseWinUI**. In the case of a WPF application, the property would be called **UseWPF**; in the case of Windows Forms, it would be **UseWindowsForms** instead.
- Lastly, every application that uses the Windows App SDK must reference the proper NuGet packages, which contain the library itself. Without these packages, you won't be able to access any of the features exposed by the runtime. The required package is called

Microsoft.WindowsAppSDK and it's a meta-package – it doesn't actually contain any code, but it groups together all the various packages that contain the various components of the framework.

From a project's structure perspective, a WinUI 3 application follows the same pattern as WPF or UWP. The default template contains the following components:

- **App.xaml**: This is the entry point of the application. It takes care of initializing everything that is needed to run the application. From a developer perspective, you can use it to store visual resources (such as styles) that must be shared across the whole application; you can use the code-behind class (the **App.xaml.cs** file) instead to manage the life cycle of the application and intercept various activation events, such as the application being opened via a custom protocol or a file type association.
- **MainWindow.xaml**: This is the main window of the application. If you take a look at the **App.xaml.cs** file, you will discover that there's an event handler called **OnLaunched()**, which creates a new instance of the **MainWindow** class and activates it. This means that when the user launches the application, the content of **MainWindow** is the first thing they will see.
- **Package.appxmanifest**: This is the manifest that describes the application and its identity. If you double-click on the file, Visual Studio gives you the opportunity to customize it through a visual interface. However, the designer was created during the Universal Windows App time frame, and as such, some of the features that we're going to explore in this book that are specific to the Win32 ecosystem aren't available. In these scenarios, you must manually edit the manifest file by right-clicking on it and choosing **View code**, which will give you direct access to the XML behind it.

If you analyze the XML, you will notice something peculiar in the **Capabilities** section:

```
<Capabilities>
  <rescap:Capability Name="runFullTrust" />
</Capabilities>
```

The MSIX packaging format, which was originally called AppX, was introduced for Windows Store apps (later evolved into UWP apps), and as such, it was tailored for applications that run inside a sandbox. This special capability, called **runFullTrust**, instead enables packages to also host traditional Win32 applications, such as a WinUI 3 application based on the .NET runtime.

When you use this template, you don't have to worry about managing the Windows App SDK runtime dependency. Even if you don't see it in the **Package.appxmanifest** file, Visual Studio will automatically inject the required entry into the final manifest of the package to install the runtime if it's not already available on the machine.

Using a separate packaging project

If you look at the available templates to create a new WinUI application, you will find one called **Blank App, Packaged with Windows Application Packaging Project (WinUI 3 in Desktop)**. This template was originally the only supported way to build and deploy WinUI applications, and it's made up of two different projects:

- One with the name you chose during the setup wizard (for example, **MyFirstApp**)
- One with the same name, plus the **(Package)** suffix (for example, **My FirstApp (Package)**):

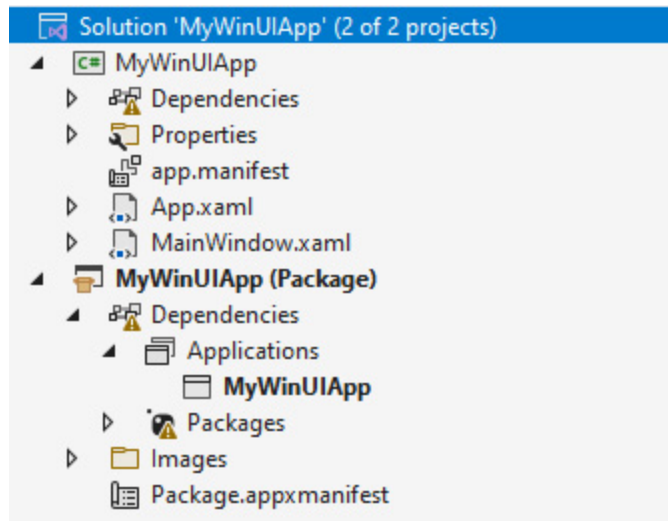


Figure 1.7 – A WinUI project that uses a separate packaging project

The first project is the main one, which contains the application. The second project, the one with the **(Package)** suffix, is a **Windows Application Packaging (WAP) Project**. This is a special type of project that doesn't contain actual code, but it enables you to package the main application with MSIX. If you expand the **Dependencies** node, you will find a section called **Applications**, which includes a reference to the main project.

This project isn't required anymore, thanks to a feature called single-project MSIX. When we create a new application using the **Blank App, Packaged (WinUI 3 in Desktop)** template that we have seen before, we're still using the MSIX packaging format. However, the WAP Project is now hidden and incorporated into the main project, thanks to a special property that you can see in the **.csproj** file:

```
<EnablePreviewMsixTooling>true</EnablePreviewMsixToolin  
g
```

However, there might be some scenarios where you still need to use the WAP Project. For example, if you're planning to bundle multiple applications inside the same MSIX package, you must continue to use a WAP Project.

A new unpackaged WinUI application

WinUI applications can also use the unpackaged model, which gives you the flexibility to distribute your application using the technology you prefer. The starting point for a WinUI unpackaged application is the same template we have used for the packaged version, the one called **Blank App, Packaged (WinUI 3 in Desktop)**. However, in this case, we must change the `.csproj` file by adding the following entry inside `PropertyGroup`:

```
<WindowsPackageType>None</WindowsPackageType>
```

This is how the full `.csproj` file should look:

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <OutputType>WinExe</OutputType>
    <TargetFramework>net6.0-windows10.0.19041.0</TargetFramework>
    <TargetPlatformMinVersion>10.0.17763.0</TargetPlatformMinVersion>
    <RootNamespace>App15</RootNamespace>
    <ApplicationManifest>app.manifest</ApplicationManifest>
    <Platforms>x86;x64;arm64</Platforms>
    <RuntimeIdentifiers>win10-x86;win10-x64;win10-arm64</RuntimeIdentifiers>
    <PublishProfile>win10-$(Platform).pubxml</PublishProfile>
    <UseWinUI>true</UseWinUI>
    <EnablePreviewMsixTooling>true</EnablePreviewMsixTooling>
    <WindowsPackageType>None</WindowsPackageType>
  </PropertyGroup>
```

Figure 1.8 – A WinUI project configured to support the unpackaged model

Thanks to this property, the application will automatically inject the required code to initialize the Windows App SDK runtime.

Once you make this change, you will notice that in the debugger drop-down menu, there will be a new entry with the name of your application followed by the **(Unpackaged)** suffix, as shown in the following screenshot:

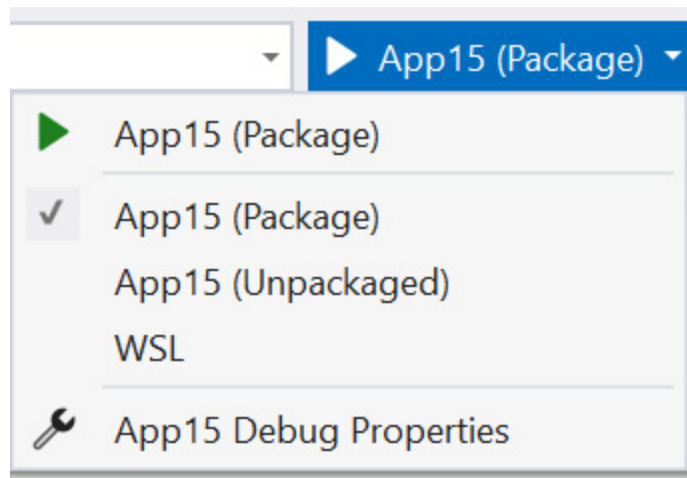


Figure 1.9 – The debugger menu once you have configured a WinUI project to run unpackaged

By choosing the unpackaged version as the target and clicking on the button (or pressing *F5*), you will launch the unpackaged WinUI application with the debugger attached.

Adding support to an existing application

All the features of the Windows App SDK other than WinUI can also be used by existing applications. In this section, we're going to learn how to set up a Windows Forms or WPF application so that it can use the Windows App SDK features.

Regardless of the UI framework of your choice, remember that the Windows App SDK targets the new .NET runtime, so you will have to start with an application that targets at least .NET 5.

Here are the steps to follow:

1. Change the **TargetFramework** property to at least **net5.0–windows10.0.19041.0** or, even better, **net6.0–windows10.0.19041.0**. If you have an existing Windows Forms or WPF application, you're already using the specific .NET runtime workload for a Windows app. You can verify this by double-clicking on your project in Visual Studio. The property should be equal

to **net5.0-windows** or **net6.0-windows**. However, this version of the workload isn't enough for our needs. This workload gives you access to generic Win32 APIs such as registry access or event log interaction, but it doesn't include the specific Windows 10 APIs introduced with UWP. As such, we must switch to one of the specific Windows 10 workloads. This is what the property must look like:

```
<TargetFramework>net6.0-windows10.0.19041.0</TargetFramework>
```

2. Install, through NuGet, the package called **Microsoft.WindowsAppSDK**, which is a meta-package; this means that it doesn't actually include any libraries, but it will properly install all the packages that are required to use the Windows App SDK. The easiest way to install it is to right-click on your project in Visual Studio, choose **Manage NuGet Packages**, and look for the package called **Microsoft.WindowsAppSDK**.
3. Specify the runtime and platforms supported by the project by adding the following entry in the **.csproj** file:

```
<Platforms>x86;x64;arm64</Platforms>
<RuntimeIdentifiers>win10-x86;win10-x64;win10-arm64</RuntimeIdentifiers>
```

This is needed, since the Windows App SDK is a native dependency, and as such, a project that uses it can't be compiled with the generic **AnyCPU** architecture, but it must target a specific CPU platform.

IMPORTANT NOTE

If you explore the content of the Windows App SDK meta-package, you might be tempted to install only a few of the sub-packages, based on the features you need. However, this isn't a supported scenario. You must always install the whole SDK, even if you aren't going to use all the available features.

The next step is different, based on the way you're going to deploy your application – packaged or unpackaged.

Distribution via a packaged app

If you want to leverage MSIX as a deployment technology, the next step is to add a WAP Project to your solution. At the time of writing, the single-project MSIX feature is supported only by WinUI apps, so you must continue using the WAP Project if you want to package an existing Windows Forms or WPF application. You can achieve this goal by right-clicking on your solution, choosing **Add | New project**, and selecting the template called **Windows Application Packaging Project**.

After you have given it a meaningful name, you must perform the following actions:

1. Select the application you want to package. To do this, right-click on the project and choose **Add | Project Reference**, and then look for your solution's project that contains the application you're looking to package.
2. Install the **Microsoft.WindowsAppSDK** package from NuGet in this project. The package will add the build steps that are required to properly generate a package – for example, thanks to this package, you won't have to manually declare in the manifest the dependency from the Windows App SDK runtime, but it will be automatically added at compile time for you.

From now on, make sure to right-click on **Windows Application Packaging Project** in **Solution Explorer** and choose **Set as startup project**. You will always need to launch and deploy this project to run and debug your application.

Distribution via an unpackaged app

Unpackaged apps use the concept of **dynamic dependencies** to dynamically take a dependency on the Windows App SDK runtime. This is achieved by using a bootstrapper API, which must be called as the

first step when an application starts so that it can properly reference the runtime and start using its APIs and components. If you're creating a new WinUI application using the dedicated template, the bootstrapper API is implicitly used when you add the **WindowsPackageType** property in a project's configuration. If you want to integrate the Windows App SDK in an existing Windows Forms or WPF application, you must instead manually invoke it. This class, called **Bootstrap**, is exposed by the **Microsoft.Windows.ApplicationModel.DynamicDependency** namespace.

Thanks to this class, you will have access to two critical methods needed to reference the Windows App SDK runtime:

- **Initialize()**: This function must be called in the app's startup, passing a reference to the version of the Windows App SDK you want to use – for example, if you want to use the stable 1.0 release, you will have to pass **0x00010000** as the value. The bootstrapper will use this information to reference the version of the framework that matches the Windows App SDK version used in the application. The function also takes care of initializing the DDLM component, which makes sure that the operating system won't try to update the framework while any unpackaged app is using it.
- **Shutdown()**: This function must be called when the user quits the application or when you don't need to use any feature provided by the Windows App SDK anymore. This function cleans up the runtime reference and shuts down the DDLM so that Windows can update the framework if needed.

The way you call these functions depends on the platform you have chosen to build your Windows application. If you have a WPF application, you can take advantage of the **OnStartup()** and **OnExit()** event handlers available in the **App** class, as in the following example:

```
using  
Microsoft.Windows.ApplicationModel.DynamicDependency;
```

```
public partial class App : Application
{
    protected override void OnStartup(StartupEventArgs e)
    {
        Bootstrap.Initialize(0x00010000);
    }
    protected override void OnExit(ExitEventArgs e)
    {
        Bootstrap.Shutdown();
    }
}
```

In Windows Forms, you can instead leverage the **Program** class and call the **Initialize()** function before **Application.Run()**, calling **Shutdown()** immediately after:

```
static class Program
{
    /// <summary>
    /// The main entry point for the application.
    /// </summary>
    [STAThread]
    static void Main()
    {
        Bootstrap.Initialize(0x00010000);
        Application.SetHighDpiMode
            (HighDpiMode.SystemAware);
        Application.EnableVisualStyles();
        Application.SetCompatibleTextRenderingDefault
            (false);
        Application.Run(new Form1());
        Bootstrap.Shutdown();
    }
}
```

Now that we have learned how to start a new project based on the Windows App SDK, let's see how we can enrich our project by creating new libraries and components.

Building libraries and components

A common requirement when you start to build more complex applications is to split a solution into multiple projects, with the goal of increasing reusability and having a clearer separation across various components.

When you're building a Windows application that uses the Windows App SDK, you have two options to choose from:

- A WinUI class library
- A .NET class library

Let's discuss each option in detail.

Using a WinUI class library

A WinUI class library is the best choice when you're building a library that contains code specific to the Windows App SDK and WinUI. It's a great fit when you want to store the following:

- Custom controls or user controls for WinUI
- Classes that use specific Windows 10 APIs

This type of class library is a great fit when you want to share code across multiple WinUI applications. To create a WinUI class library, you can use the template in Visual Studio called **Class Library (WinUI 3 in Desktop)**.

Using a .NET class library

A .NET class library is the best choice when you're building a library that doesn't include any code that is specific to the Windows App SDK. By using a generic .NET class library, you can share it not only with other applications based on the Windows App SDK but also with every other project type supported by .NET – web apps, cloud services, mobile apps, and so on.

When you choose the **Class library** project type in Visual Studio, there are two options to choose from:

- A specific .NET version (for example, .NET 5 or .NET 6)
- .NET Standard

The first option is the best choice when you're planning to share this library with any other project that is using the new .NET runtime. This choice will give you access to the widest surface of APIs and features, such as the latest additions to the C# language.

Alternatively, .NET Standard is the best choice when you're going to share this library with older platforms that are based on other flavors of the .NET runtime that were created before the unification journey started with .NET 5. In fact, .NET Standard was created when developers had the need to share code across different implementations of .NET, such as Mono (used by Xamarin and the first release of Blazor), .NET Core, and the full .NET Framework. .NET Standard defines a set of APIs through a contract, which must be agreed by all the various implementations of the platform. When a platform implements a specific revision of a contract, it means that it implements all the APIs that are part of the revision.

The currently most widely adopted revisions of .NET Standard are 2.0 and 2.1:

- 2.0 is the best choice when you need to share your code with .NET Framework applications.
- 2.1 is the best choice when supporting .NET Framework isn't a requirement but you need to share your code with a .NET Core or Xamarin application.

.NET Standard has played a critical role in the .NET ecosystem in the past. However, in the long-term future, as developers will gradually move their .NET projects to the new .NET runtime introduced with .NET 5, it won't be required anymore. You will just need to target the lowest version of the .NET runtime across all your projects. For example, if you need to share code between a WinUI application based on .NET 6, a web project based on .NET 7, and a mobile application based on .NET 8, it will be enough to create a class library that targets .NET 6.0 to share it across all of them.

Summary

We started this chapter by learning the fundamental concepts that will guide us through the book. We learned how the Windows App SDK works, how it compares to the other development frameworks, and how you can adopt it in your projects.

Throughout this chapter, we learned what the Windows App SDK is and how we can use it to build new projects or integrate it into existing Windows applications. Then, we reviewed why the new .NET runtime is the future for Windows developers who want to build applications using Visual Studio and C#. We also learned which deployment models you can adopt to distribute your applications.

With this, we're ready to start our modernization journey and to start using all the features that the Windows App SDK has to offer.

In the next chapter, we will learn about the differences between Windows Forms and the Windows App SDK and start with the basics.

Questions

1. Can you integrate the Windows App SDK into an existing Windows Forms application based on .NET Framework?
2. Which is the best technology to deploy a Windows application that uses the Windows App SDK?
3. Is WinUI 3 the only feature included in the Windows App SDK?

Further reading

- *Learn WinUI 3.0, Alvin Ashcraft*
- *C# 9 and .NET 5: Modern Cross-Platform Development – Fifth Edition, Mark J. Price*